KIẾN TRÚC HỆ THỐNG

Hệ thống ghép cặp CV – Việc làm theo section
(Section-Aware CV–Job Matching) kèm hàng đợi xử lý file nền

Nhóm nghiên cứu: .....
Giáo viên hướng dẫn: .....

October 21, 2026

Abstract

Tài liệu mô tả kiến trúc phần mềm của hệ thống ghép cặp "sơ yếu lý lịch (CV) – việc làm (JD)" theo từng *phần (section)* của văn bản, gồm hai tiến trình lớn: (1) **AI service** viết bằng Python/FastAPI, đảm nhận trích xuất văn bản, phân tích bằng mô hình ngôn ngữ lớn (LLM), sinh embedding theo section, lưu vector và xếp hạng hybrid (semantic + từ khóa + ràng buộc có cấu trúc); và (2) **API chính** viết bằng NestJS, đảm nhận nghiệp vụ người dùng và *gọi* AI service. Trọng tâm của tài liệu là **hàng đợi xử lý file nền** (background processing queue) cho phép người dùng upload CV và nhận trạng thái xử lý từng bước thay cho lời gọi đồng bộ "treo" hàng chục giây. Tài liệu gồm: sơ đồ tổng thể, sơ đồ lớp module, máy trạng thái job, sơ đồ chuỗi lời gọi (bước upload bất đồng bộ), mô hình dữ liệu vector theo section và luồng xếp hạng hybrid; phần cuối nêu hướng nghiên cứu khoa học đề xuất.

Contents

1 Tổng quan hệ thống **2**

2 Kiến trúc bên trong AI service **2**

3 Hàng đợi xử lý file nền (phần lõi) **4**
3.1 Vấn đề . . . 4
3.2 Máy trạng thái của job . . . 4
3.3 Luồng bất đồng bộ khi upload CV . . . 4
3.4 Độ tin cậy . . . 4

4 Mô hình dữ liệu vector theo section **5**

5 Luồng xếp hạng hybrid **6**
5.1 Công thức chính . . . 7

6 Hợp đồng API (tóm tắt) **7**

7 Cấu hình & công nghệ **7**

8 Kiểm thử & đánh giá **7**

9 Đề xuất hướng nghiên cứu khoa học **8**
9.1 Câu hỏi nghiên cứu . . . 8
9.2 Ma trận thí nghiệm đề xuất . . . 8
9.3 Dữ liệu & quy trình . . . 8
9.4 Lộ trình 12 tuần (đề xuất) . . . 9
9.5 Rủi ro & biện pháp . . . 9
9.6 Tài liệu tham khảo chính . . . 9

1 Tổng quan hệ thống

Hệ thống có hai tiến trình chính (hai mã nguồn khác nhau) và một số thành phần hạ tầng:

- **API chính (NestJS, TypeScript)** — backend nghiệp vụ: xác thực, hồ sơ người dùng, tin tuyển dụng, hợp đồng, thanh toán. Là nơi duy nhất được phép đọc/ghi **PostgreSQL** (nguồn dữ liệu gốc).
- **AI service (Python, FastAPI)** — microservice nội bộ, *không* có quyền ghi PostgreSQL: trích xuất PDF/DOCX, phân tích cấu trúc bằng `qwen3:4b` (chạy local qua Ollama), sinh embedding `BAAI/bge-m3`, lưu vector vào **ChromaDB**, xếp hạng và giải thích bằng LLM.
- **ChromaDB** — cơ sở vector (HNSW, cosine) persistent trên đĩa.
- **SQLite** — bảng `processing.db` lưu *trạng thái job xử lý file* (hàng đợi nền). Lý do dùng SQLite: ở giai đoạn này AI service chỉ được cấp quyền *đọc* PostgreSQL; giao diện `ProcessingStore` được thiết kế để sau này thay bằng `PostgresProcessingStore` mà không đổi pipeline.
- **Outbox + `sync_worker`** — cơ chế đồng bộ *một chiều* PostgreSQL → Chroma: các bản ghi CV/JD trong DB được đẩy sang vector store để phục vụ truy hồi.

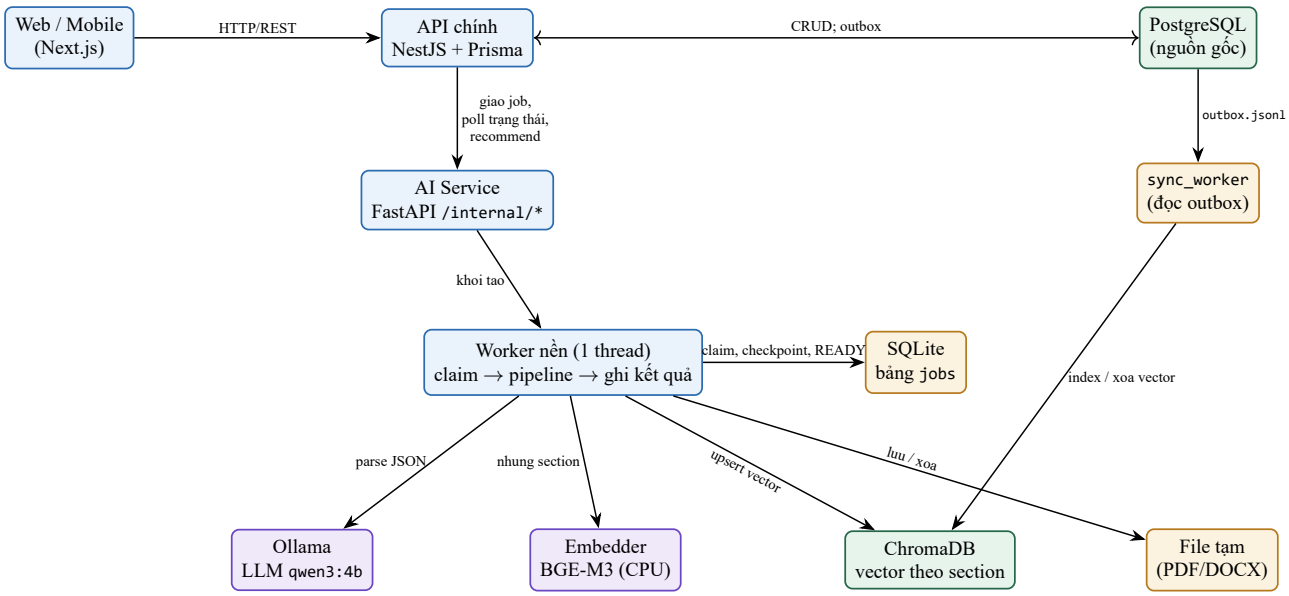


Figure 1: Tổng quan hệ thống: API chính (NestJS) điều phối, AI service (FastAPI) xử lý và xếp hạng, dữ liệu vector nằm ở ChromaDB, trạng thái job nằm ở SQLite.

Hai thành phần *giao tiếp* với nhau chỉ qua HTTP nội bộ, được bảo vệ bằng `INTERNAL_API_KEY` (che dấu `/internal/*` khỏi Internet). Nhờ đó AI service có thể thay đổi độc lập (thêm model, thêm queue) mà không ảnh hưởng API chính.

2 Kiến trúc bên trong AI service

AI service được chia thành bốn lớp phụ trách rõ ràng (Hình 2), theo nguyên tắc *API chỉ điều phối, nghiệp vụ nằm ở lớp giữa, lớp dưới lo hạ tầng*. Thư viện lõi `src/cvjd` (Python package `cvjd`) độc lập với web framework để có thể chạy benchmark từ dòng lệnh.

Hai đường xử lý "đồng bộ" và "nền" dùng chung các khối nghiệp vụ nên kết quả phân tích là như nhau (Hình 3).

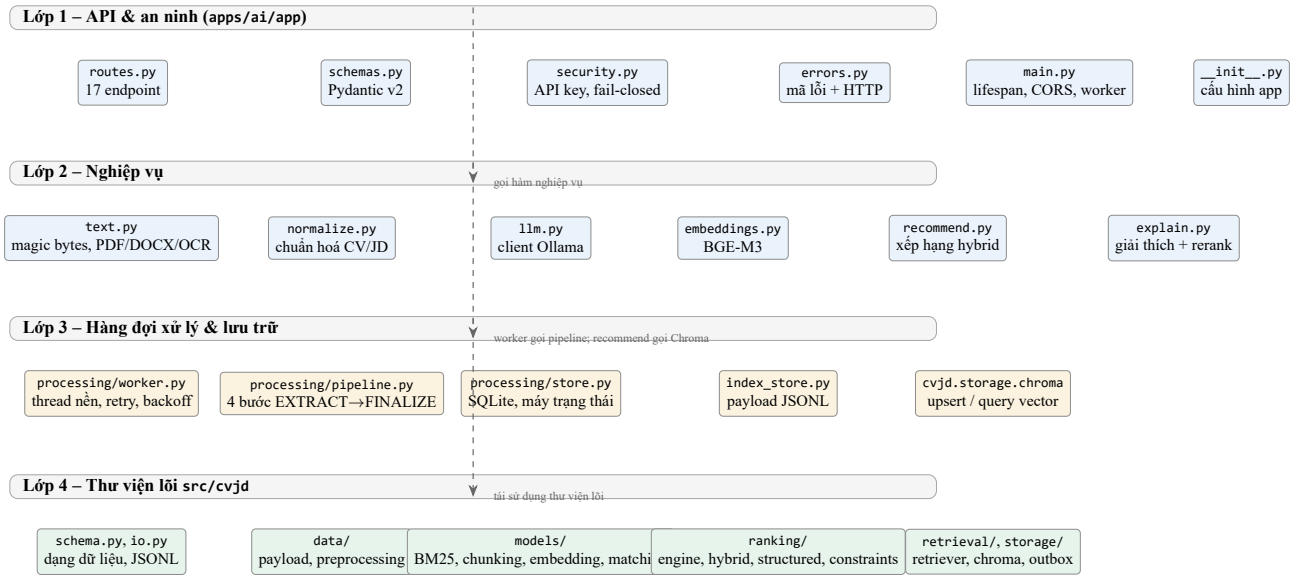


Figure 2: Các lớp module của AI service. Lớp ngoài tiếp nhận HTTP, lớp nghiệp vụ xử lý văn bản/LLM, lớp hàng đợi lo việc chạy nền, thư viện lõi chứa các mô hình và thứ tự xếp hạng (có thể chạy riêng từ CLI để benchmark).

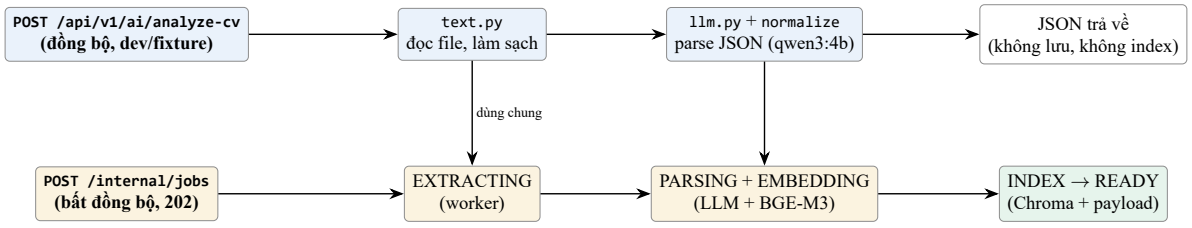


Figure 3: Hai đường xử lý dùng chung khối trích xuất và LLM: đường đồng bộ `analyze-cv` chỉ trả JSON, đường hàng đợi còn nhúng và index vector để phục vụ truy hồi sau đó.

3 Hàng đợi xử lý file nền (phần lõi)

3.1 Vấn đề

Việc phân tích một CV mất 10–40 giây (trích xuất PDF, gọi LLM sinh JSON, nhúng 5–8 section). Nếu để HTTP đồng bộ "treo" chừng đó thời gian thì trình duyệt/người dùng dễ bị timeout và NestJS cũng bị chiếm kết nối. Giải pháp: **hàng đợi job nền**

1. Client gửi file → nhận ngay `202 Accepted` kèm `job_id`;
2. Worker chạy nền xử lý từng job, cập nhật *step* + *progress*;
3. Client poll trạng thái và hiển thị "Đang trích xuất → Đang phân tích → Đang nhúng → Hoàn tất".

3.2 Máy trạng thái của job

Mỗi job có *trạng thái* (`status`) và *bước đang chạy* (`step`) là hai chiều độc lập: trạng thái nói "job đang đi tới đâu về mặt vòng đời", bước nói "đang ở khâu nào" để hiển thị tiến độ (Hình 4).

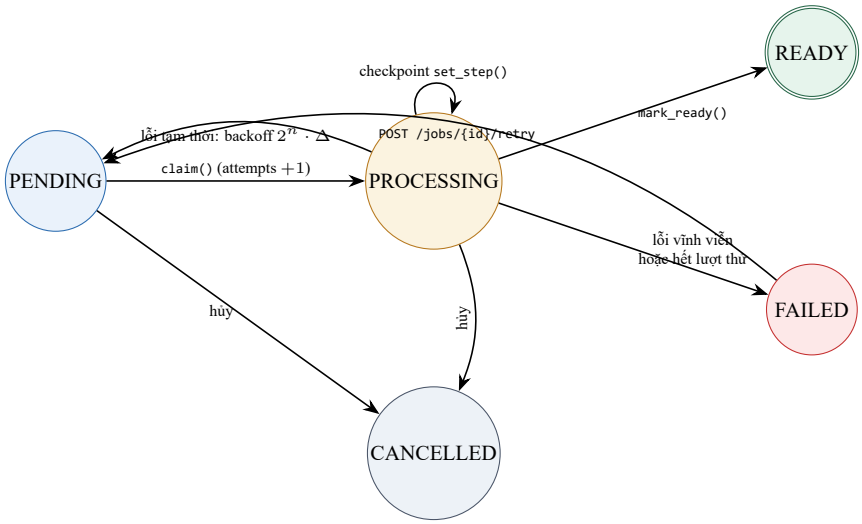


Figure 4: Máy trạng thái job xử lý file. `READY` là trạng thái hấp thụ (job hoàn tất); `FAILED` vẫn giữ file gốc để có thể retry thủ công.

Bên trong `PROCESSING`, pipeline chạy tuần tự bốn bước, mỗi bước ghi *checkpoint* vào DB nên nếu service chết giữa chừng thì ta biết chính xác job đang ở đâu:

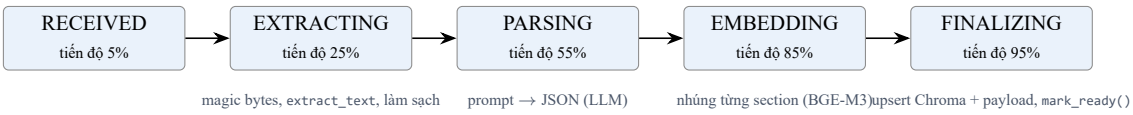


Figure 5: Các bước bên trong `PROCESSING` và mức tiến độ tương ứng được ghi vào `step`/`progress`.

3.3 Luồng bất đồng bộ khi upload CV

Hình 6 mô tả toàn bộ lời gọi khi người dùng upload CV: từ HTTP request đến khi job chuyển sang `READY` và client poll được kết quả.

3.4 Độ tin cậy

- **Phân loại lỗi:** lỗi *vĩnh viễn* (file hỏng, text rỗng, sai định dạng) → `FAILED` ngay, không thử lại; lỗi *tạm thời* (Ollama timeout, Chroma khóa database) → worker thử lại với backoff $2^n \cdot \Delta$ đến `JOB_MAX_ATTEMPTS` (mặc định 3).

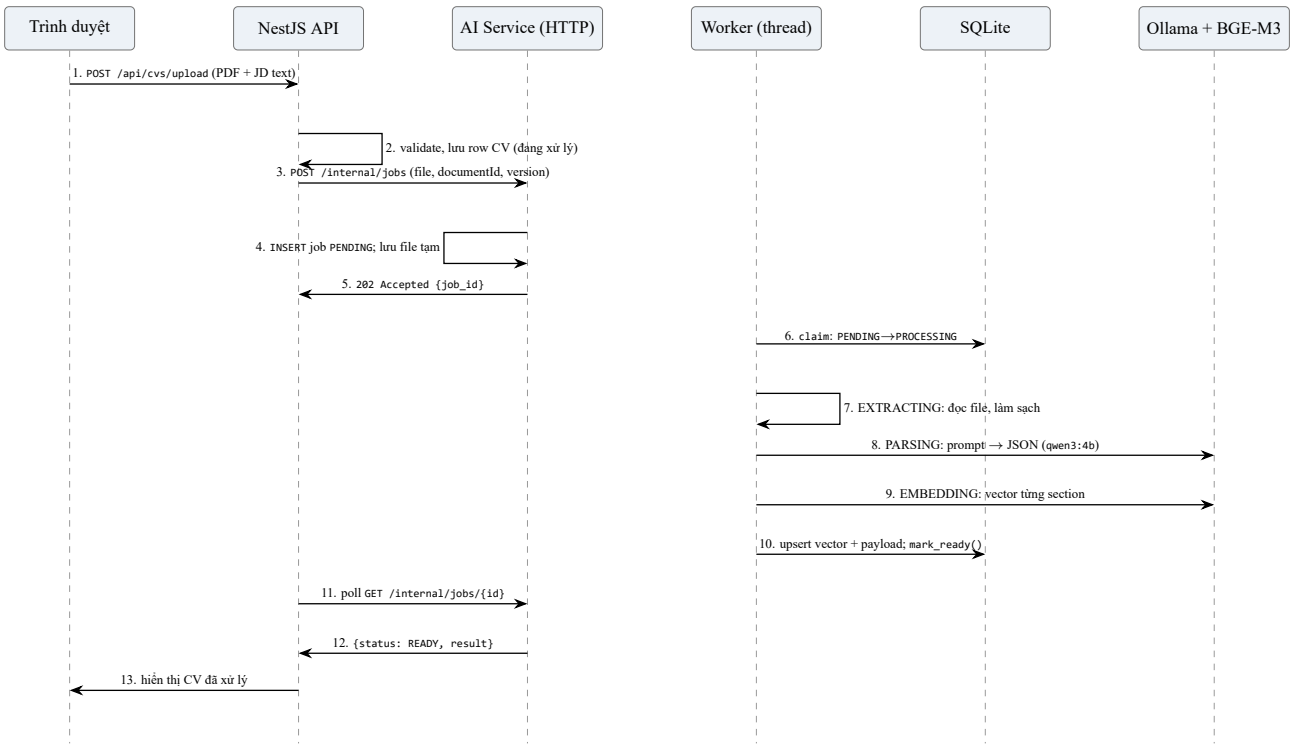


Figure 6: Chuỗi lời gọi của luồng upload bất đồng bộ. Toàn bộ bước 6–10 chạy nền trong cùng process FastAPI; API chính chỉ poll trạng thái.

- **Phát hiện job "kẹt":** nếu `PROCESSING` quá `JOB_STALE_SECONDS` (300 s) mà không cập nhật thì worker tự đưa job về `PENDING`; endpoint `POST /internal/jobs/reconcile` cho phép vận hành viên gỡ kẹt thủ công.
- **Không mất job khi restart:** SQLite là nguồn sự thật duy nhất; khi khởi động lại service, các job còn `PENDING` trong DB sẽ được worker xử lý tiếp.
- **Bỏ qua nhúng lại (idempotent):** nếu nội dung text không đổi (`content_hash` khớp) *và* vector của phiên bản đó vẫn còn nguyên trong Chroma + payload còn trong store thì job chuyển thẳng sang `READY` với cảnh báo `REUSED_INDEX` — retry không tiêu tốn thêm lượt LLM.
- **Quan sát được:** `GET /internal/jobs/stats` trả về số job theo trạng thái, độ sâu hàng đợi, số lượt retry và phân vị thời gian xử lý; `GET /readyz` báo trạng thái worker cho giám sát.

4 Mô hình dữ liệu vector theo section

Điểm đặc biệt của hệ thống là mỗi tài liệu *không* được nhúng thành một vector duy nhất mà tách theo section (kỹ năng, kinh nghiệm, dự án, học vấn, tóm tắt) cộng thêm một vector cho toàn văn. Nhờ đó khi so một JD với một CV, ta so được "yêu cầu kỹ năng" trùng với "phần kỹ năng" thay vì so toàn văn mơ hồ.

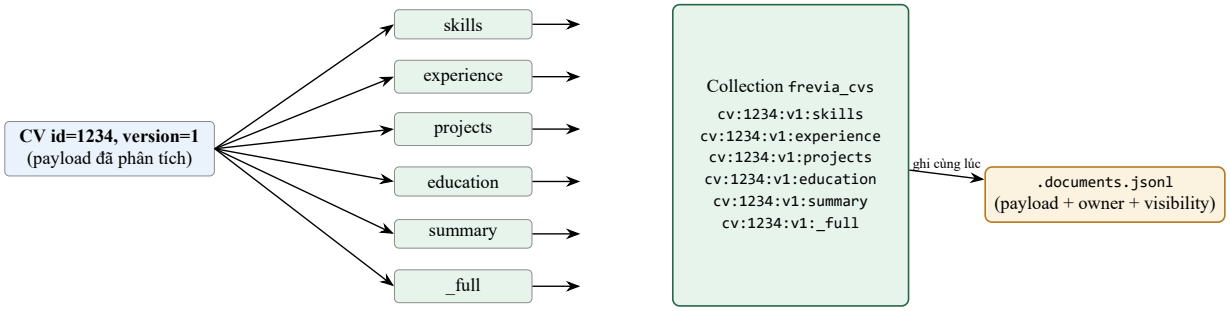


Figure 7: Mỗi CV/JD sinh ra một "bó" vector theo section. Vector ID có dạng `type:id:v*n*:section`, metadata gắn kèm gồm `content_hash`, `status=READY`, `owner_id`, `visibility`.

Khi phiên bản tài liệu tăng (`version`+1) thì upsert sẽ tự xoá các vector cũ cùng `document_id` và thay bằng bộ mới; metadata `status=READY` đảm bảo chỉ vector đã hoàn chỉnh mới được dùng cho truy hồi.

5 Luồng xếp hạng hybrid

Khi NestJS gọi `POST /internal/recommend`, AI service thực hiện: truy hồi top-P ứng viên từ Chroma, tính điểm semantic theo section, gộp với điểm từ khóa BM25, cộng điểm theo ràng buộc có cấu trúc, rồi (tuỳ chọn) để LLM chấm lại và giải thích.

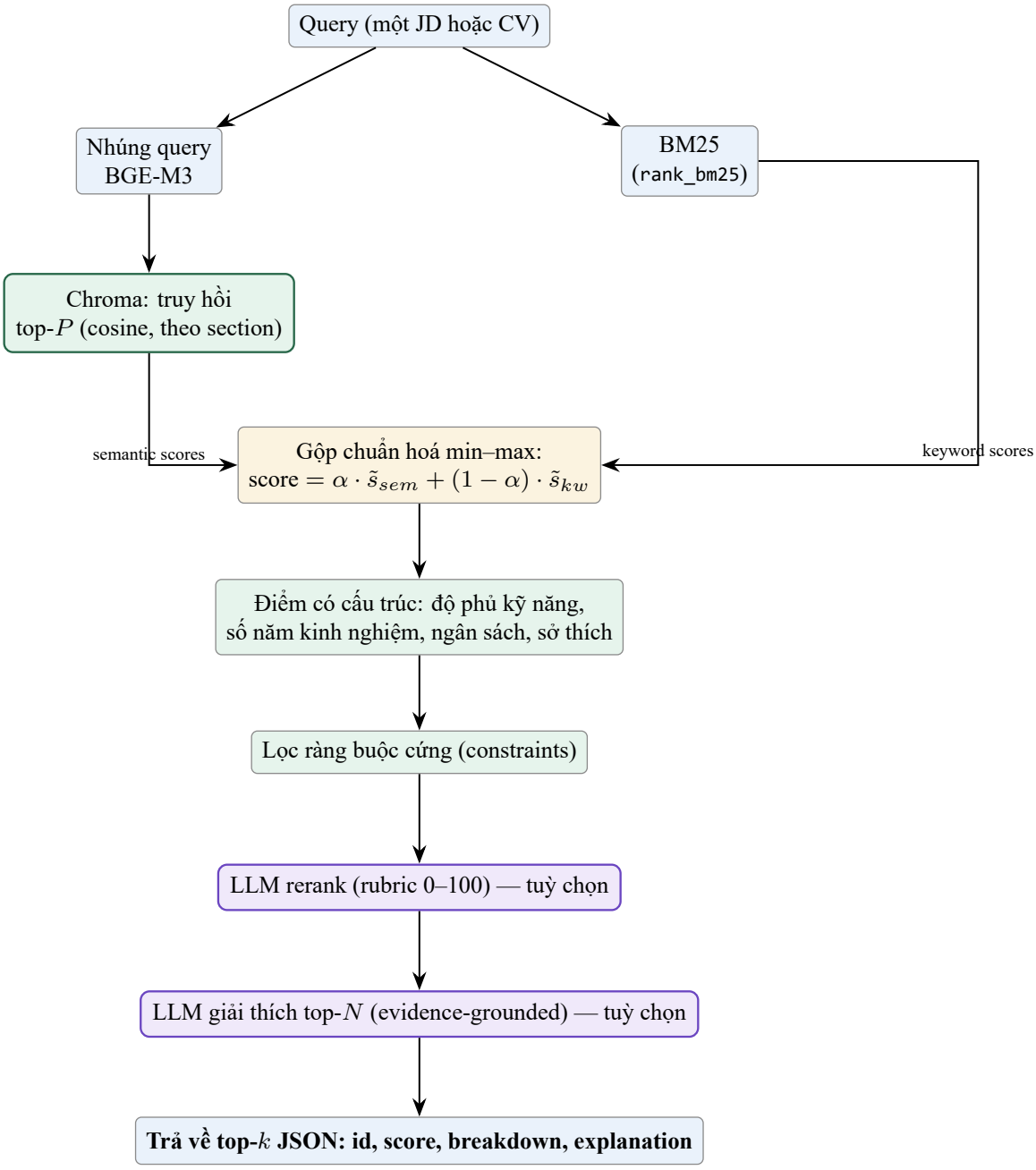


Figure 8: Luồng xếp hạng hybrid. Các bước cuối (rerank, explain) là tuỳ chọn vì mỗi ứng viên tốn thêm một lượt gọi LLM.

5.1 Công thức chính

$$s_{\text{sem}}(q,d) = \text{agg}_{x \in S_q} \max_{y \in S_d} \cos\big(e(x),\, e(y)\big) \tag{1}$$

$$s(q,d) = \alpha \cdot \widetilde{s}_{\text{sem}}(q,d) + (1-\alpha) \cdot \widetilde{s}_{\text{kw}}(q,d) \tag{2}$$

Trong đó S_q, S_d là tập section của truy vấn và tài liệu, $e(\cdot)$ là vector BGE-M3, agg là phép gộp theo section (`mean` hoặc `max`), hàm $\widetilde{\cdot}$ là chuẩn hoá min–max trên tập ứng viên; α là tham số pha trộn (có thể ghi đè theo từng request). Hai phương án đối chiếu là *single-vector* (một vector toàn văn, $|S| = 1$) và *BM25 thuần* ($\alpha = 0$).

6 Hợp đồng API (tóm tắt)

Method & Path	Trạng thái	Vai trò
`POST /api/v1/ai/analyze-cv`	200	Phân tích CV đồng bộ (trả JSON, không lưu)
`POST /internal/extract-cv`	200	Trích xuất + phân tích CV (nội bộ)
`POST /internal/extract-job`	200	Trích xuất + phân tích JD (nội bộ)
`POST /internal/index`	200	Nhúng + upsert vector cho danh sách payload
`POST /internal/index/delete`	200	Xoá vector theo id/type
`POST /internal/recommend`	200	Xếp hạng hybrid (kèm `explain`)
`POST /internal/explain`	200	Giải thích 1 cặp (job, cv) bằng LLM
`POST /internal/rerank`	200	Chấm lại theo rubric 0–100
`POST /internal/embeddings`	200	Sinh vector thô cho danh sách chuỗi
`POST /internal/jobs`	202	Tạo job xử lý file (multipart) → `job_id`
`GET /internal/jobs/{id}`	200	Trạng thái + tiến độ + cảnh báo
`DELETE /internal/jobs/{id}`	200	Hủy job chưa xong
`POST /internal/jobs/{id}/retry`	200	Đưa job `FAILED` về `PENDING`
`GET /internal/jobs/stats`	200	Số job theo trạng thái, hàng đợi, phân vị thời gian
`POST /internal/jobs/reconcile`	200	Gỡ job kẹt (idempotent)
`GET /healthz`, `GET /readyz`	200	Sống/khỏe (ping Ollama, độ sâu hàng đợi)

Table 1: Các endpoint của AI service. Mọi đường `/internal/*` đều bắt buộc `X-Internal-Api-Key` khi bật `INTERNAL_API_KEY_REQUIRED=1`.

Mẫu phản hồi tiêu biểu của job (rút gọn):

```
{
  "job_id": "j_9f2c...", "status": "PROCESSING",
  "document_type": "cv", "document_id": "1234", "version": 3,
  "step": "PARSING", "progress": 55,
  "attempt_count": 1, "warnings": [],
  "duration_ms": null, "result": null
}
```

7 Cấu hình & công nghệ

Các biến môi trường quan trọng của hàng đợi (mặc định trong dấu ngoặc):

8 Kiểm thử & đánh giá

- **Kiểm thử tự động:** 138 test pytest bao phủ: trích xuất văn bản, chuẩn hoá payload, xếp hạng hybrid, giải thích/rerank (LLM giả lập), máy trạng thái job, worker (retry, backoff, job kẹt, hủy), hợp đồng API. Chroma dùng *ephemeral client* + `MockEmbedder` nên test chạy nhanh, không tải model thật.

Nhóm	Thành phần	Lựa chọn	Lý do
Web frontend	Framework	Next.js	(repo riêng)
API nghiệp vụ	Backend	NestJS + Prisma	(repo riêng)
AI service	Ngôn ngữ	Python 3.13 + FastAPI	Async, OpenAPI tự sinh
LLM	Mô hình	`qwen3:4b` (Ollama)	Không mất dữ liệu ra ngoài
Embedding	Mô hình	`BAAI/bge-m3` (1024 chiều, CPU)	Đa ngôn ngữ, nhiều chức năng
Vector store	Cơ sở	ChromaDB persistent (HNSW, cosine)	Nhúng thư viện, đơn giản
Job store	Cơ sở	SQLite (WAL)	PG đang chỉ đọc; interface để chuyển Postgres
Từ khóa	Thuật toán	BM25 (`rank_bm25`)	Baseline kinh điển, giải thích được
Kiểm thử	Công cụ	pytest, ruff	138 test tự động

Table 2: Công nghệ chính của AI service.

Biến	Mặc định	Vai trò
`OLLAMA_MODEL`	– (bắt buộc)	Mô hình LLM dùng cho parse/explain/rerank
`EMBED_MODEL`	`BAAI/bge-m3`	Mô hình sinh embedding
`CHROMA_PATH`	`None`	Nơi lưu Chroma; `None` = bộ nhớ (dùng cho test)
`INTERNAL_API_KEY_REQUIRED`	`0`	Bật = "fail-closed" khi chưa cấu hình key
`PROCESSING_WORKER_ENABLED`	`0`	Bật worker nền trong process FastAPI
`PROCESSING_DB_PATH`	`data/processing.db`	Đường dẫn SQLite của hàng đợi
`PROCESSING_FILES_DIR`	`data/processing_files`	Thư mục file tạm
`JOB_MAX_ATTEMPTS`	3	Số lượt thử tối đa cho lỗi tạm thời
`JOB_STALE_SECONDS`	300	Ngưỡng coi job `PROCESSING` là kẹt
`JOB_RETRY_BASE_DELAY_S`	1.0	Hệ số backoff $2^n \cdot \Delta$
`PROCESSING_SKIP_REEMBED`	1	Bật bỏ qua nhúng lại khi nội dung không đổi

Table 3: Cấu hình hàng đợi xử lý (xem đầy đủ tại `apps/ai/.env.example`).

- **Kiểm soát chất lượng mã:** `ruff` cho mã mới; tài liệu thiết kế (`docs/DESIGN_processing_queue.md`) được cập nhật đồng bộ với mã.
- **Đánh giá chất lượng ghép cặp:** dùng bộ chỉ số chuẩn của truy hồi thông tin — Precision@k, Recall@k, nDCG@k, MRR, MAP — trên tập có nhãn (xem mục sau).

9 Đề xuất hướng nghiên cứu khoa học

9.1 Câu hỏi nghiên cứu

1. **RQ1 — Giá trị của việc tách theo section:** xếp hạng theo multi-vector từng section có tốt hơn nhúng toàn văn (single-vector) và BM25 thuần hay không?
2. **RQ2 — Pha trộn semantic – từ khóa:** giá trị α nào tối ưu (`eq. 2`) trên dữ liệu tiếng Việt/kép Việt – Anh?
3. **RQ3 — Vai trò của LLM ở cuối pipeline:** rerank theo rubric và giải thích evidence-grounded có cải thiện thứ tự xếp hạng và độ tin cậy của người dùng không (đo bằng LLM-as-judge và khảo sát người thật)?
4. **RQ4 — Chi phí – hiệu năng:** hàng đợi nền với bỏ qua nhúng lại (`content_hash`) tiết kiệm bao nhiêu thời gian/token so với xử lý đồng bộ và xử lý lại toàn bộ?

9.2 Ma trận thí nghiệm đề xuất

9.3 Dữ liệu & quy trình

- Bộ dữ liệu công khai (Kaggle: *Resume Dataset*, *Job Descriptions*) và dữ liệu nội bộ đã có sẵn pipeline đồng bộ từ PostgreSQL sang Chroma; yêu cầu tối thiểu 500 cặp CV–JD có nhãn "phù hợp / không phù hợp".
- Chia $70/10/20$ (train/dev/test); tham số α, pool, ngưỡng lọc ràng buộc được tinh chỉnh trên dev, chỉ chốt một lần trên test.

Nhóm biến	Tùy chọn	Chỉ số
Phương pháp	BM25; single-vector; section-aware (mean/max); chunk max-sim	nDCG@10, MRR
Tham số	$\alpha \in \{0, 0.3, 0.5, 0.7, 1.0\}$; pool $P \in \{50, 100, 200\}$	nDCG@10, độ trễ
Embedding	bge-m3; multilingual-e5-small; MiniLM đa ngôn ngữ	nDCG@10, thời gian nhúng
Tầng LLM	không rerank; rerank; rerank + explain	MRR, điểm LLM-as-judge (1–5)
Hàng đợi	đồng bộ; nền toàn bộ; nền + skip re-embed	thời gian phản hồi, token/ job

Table 4: Ma trận thí nghiệm (mỗi cấu hình chạy 3 lần, lấy trung bình).

- Đánh giá con người: 3–5 người đánh giá top-10 gợi ý theo thang 1–5 cho *mức độ phù hợp* và *mức độ hữu ích của phần giải thích*.

9.4 Lộ trình 12 tuần (đề xuất)

Tuần	Giai đoạn	Nội dung
1–2	Chuẩn bị	Đọc tài liệu, dựng bộ dữ liệu có nhãn, hoàn tất baseline BM25
3–5	Thí nghiệm	Chạy section-aware vs single-vector (RQ1), tinh chỉnh α (RQ2)
6–8	LLM	Thí nghiệm rerank + explain, đánh giá LLM-as-judge (RQ3)
9–10	Hệ thống	Đo chi phí hàng đợi vs đồng bộ, đo độ trễ đầu-cuối (RQ4)
11–12	Viết	Viết báo cáo, vẽ biểu đồ, chuẩn bị slide thuyết trình

Table 5: Lộ trình nghiên cứu đề xuất.

9.5 Rủi ro & biện pháp

Rủi ro	Biện pháp
Thiếu dữ liệu có nhãn	Dùng dữ liệu công khai + gán nhãn thủ công theo mẫu rubric; dùng LLM-assisted labeling rồi kiểm tra chéo 20%
LLM chấm không nhất quán	Chạy 3 lần, tính đồng thuận (kappa); chốt prompt cố định
Ollama/BGE-M3 chậm trên CPU	Giữ worker 1 thread, đo riêng từng bước, dùng pool nhỏ; nếu cần chuyển sang GPU/API embedding
Kết quả không có ý nghĩa thống kê	Kiểm định Wilcoxon signed-rank giữa các cấu hình

Table 6: Rủi ro dự kiến.

9.6 Tài liệu tham khảo chính

- Chen, J. et al. (2024). *BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation*. ACL Findings.
- Robertson, S. & Zaragoza, H. (2009). *The Probabilistic Relevance Framework: BM25 and Beyond*. Foundations and Trends in IR.
- Karpukhin, V. et al. (2020). *Dense Passage Retrieval for Open-Domain Question Answering*. EMNLP.
- Nogueira, R. & Cho, K. (2019). *Passage Re-ranking with BERT*. arXiv:1901.04085.
- Zheng, L. et al. (2023). *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. NeurIPS Datasets and Benchmarks.
- Tài liệu thiết kế nội bộ: `docs/DESIGN_processing_queue.md`, `docs/WORKFLOW.md`, `plan.md` (trong repo).

Phụ lục: cấu trúc thư mục liên quan

```
section-aware-cv-job-matching/
├── apps/ai/                      # AI service (FastAPI)
│   ├── app/
│   │   ├── main.py               # lifespan, worker thread, /healthz, /readyz
│   │   ├── routes.py             # 17 endpoint (analyze-cv, /internal/*)
│   │   ├── schemas.py            # Pydantic models
│   │   ├── security.py           # API key + fail-closed
│   │   ├── text.py               # PDF/DOCX/OCR, magic bytes, làm sạch
│   │   ├── llm.py                # client Ollama (retry, timeout)
│   │   ├── embeddings.py         # BGE-M3 (+ MockEmbedder cho test)
│   │   ├── normalize.py          # chuẩn hoá CV/JD -> payload
│   │   ├── recommend.py          # xếp hạng hybrid, index, explain
│   │   ├── explain.py            # giải thích + rerank bằng LLM
│   │   ├── index_store.py        # payload JSONL cạnh Chroma
│   │   └── processing/           # hang doi nen
│   │       ├── store.py          # SQLite + may trang thai job
│   │       ├── pipeline.py       # 4 buoc EXTRACTING -> FINALIZING
│   │       └── worker.py         # thread nen, retry, backoff, stale
│   ├── tests/                    # 135 test pytest
│   └── .env.example
├── src/cvjd/                     # thu vien loi (chay duoc tu CLI)
│   ├── schema.py, io.py
│   ├── data/                     # payload, preprocessing
│   ├── models/                   # bm25, chunking, embedding, matching
│   ├── ranking/                  # engine, hybrid, structured, constraints
│   ├── retrieval/                # retriever
│   └── storage/                  # chroma, postgres, outbox
└── docs/
    ├── DESIGN_processing_queue.md
    ├── WORKFLOW.md
    └── kien_truc_he_thong.tex    # tai lieu nay
```